

Execution Manager – Data Structure Reference

This module coordinates **TaskGraph** execution. It loads two JSON files:

1. a **data** file describing nodes and runtimes, and
2. a **flow** file describing the execution order.

It also integrates with the **Namespace Manager** to snapshot MATLAB workspace state around each runtime execution.

1) Loaded JSON Files

1.1 **data** JSON (loaded into `self.data`)

Minimal fields required **by this module**:

```
{
  "nodes": [ /* array of node objects (not directly used here, but kept for
context) */ ],
  "runtimes": [
    {
      "EditorID": "string (unique)",
      "current_node": {
        "type": "MR" | "OP"
        /* other fields allowed but not required by this module */
      }

      /* optional fields for downstream usage (not required here):
      "inputs": {...},
      "params": {...},
      "tool": {...},
      "metadata": {...}
      */
    }
  ]
}
```

Notes

- **EditorID** must be **unique**; it is used as the primary key in the runtime map.
- `current_node.type` is the only field **directly** consumed by this module to branch logic:
 - "MR" → mark as available (no execution)
 - "OP" → execute (surrounded by start/end snapshots)

Example

```
{
  "nodes": [
    { "EditorID": "MR_1", "type": "MR", "label": "Raw file name" },
    { "EditorID": "OP_1", "type": "OP", "label": "Open SXM" }
  ],
  "runtimes": [
    {
      "EditorID": "MR_1",
      "current_node": { "type": "MR" },
      "metadata": { "desc": "Provide file name(s)" }
    },
    {
      "EditorID": "OP_1",
      "current_node": { "type": "OP" },
      "tool": { "name": "opensxm_auto" },
      "params": { "path": "Data", "filename": "07_15_25_002.sxm", "pnum": "1" }
    }
  ]
}
```

1.2 flow JSON (loaded into `self.flow`)

Minimal fields required **by this module**:

```
{
  "runtime_sequence": ["<runtime EditorID>", "..."]
}
```

Notes

- The array defines the exact execution order of runtimes by their `EditorID`.
- All IDs listed here must exist in `data.runtimes[*].EditorID`.

Example

```
{
  "runtime_sequence": ["MR_1", "OP_1"]
}
```

2) In-Memory Structures (created by the module)

2.1 `self.nodes`: list

Copied from `self.data.get("nodes", [])`.

This module does not read fields inside `nodes`.

2.2 `self.runtimes: list`

Copied from `self.data.get("runtimes", [])`.

Each item must contain at least:

```
{
  "EditorID": "string",
  "current_node": { "type": "MR" | "OP" }
}
```

2.3 `self.runtime_sequence: list[str]`

Copied from `self.flow.get("runtime_sequence", [])`.

2.4 `self.runtimes_by_editor_id: dict[str, dict]`

A dictionary for O(1) lookup by `EditorID`:

```
{
  "<EditorID>": {
    "EditorID": "<EditorID>",
    "current_node": { "type": "MR" | "OP" },
    "...": "other optional fields"
  }
}
```

Example

```
{
  "MR_1": {
    "EditorID": "MR_1",
    "current_node": { "type": "MR" },
    "metadata": { "desc": "Provide file name(s)" }
  },
  "OP_1": {
    "EditorID": "OP_1",
    "current_node": { "type": "OP" },
    "tool": { "name": "opensxm_auto" },
    "params": { "path": "Data", "filename": "07_15_25_002.xml", "pnum": "1" }
  }
}
```

3) Execution-Time Data Contracts

3.1 `ExecutionManager.execute_flow()`

- Iterates over `self.runtime_sequence` and calls `execute_runtime(EditorID)` unless the runtime is already "running" or "success" per **Namespace Manager** status.
- **Important:** the current code `returns` when it encounters a "running" or "success" runtime, which stops the entire loop. If you intend to skip just that runtime and continue, replace `return` with `continue`.

3.2 `ExecutionManager.execute_runtime(runtime_edid: str)`

- Retrieves `runtime_info = self.get_runtime_by_editor_id(runtime_edid)`.
- Reads `node_type = runtime_info["current_node"]["type"]`.
- Branches:
 - **MR:** Only marks as ready (no tool call, no snapshot).
 - **OP:**
 1. `namespace_mgr.start_runtime(runtime_edid)` → stores start snapshot.
 2. **TODO:** gather inputs → build meta → call tool (e.g., `call_matlab_meta(meta)`).
 3. `namespace_mgr.end_runtime(runtime_edid)` → stores end snapshot and diff.
- On exception:

```
self.namespace_mgr.runtime_outputs[runtime_edid] = {"error": str(e)}
```

This means `runtime_outputs[runtime_edid]` can be **either**:

- a **diff object** (normal case), or
- an **error object** `{"error": "<message>"}` (failure case).

Union type summary

```
type RuntimeOutput =  
  | { [varName: string]: { class: string; size: string; status: "added" |  
    "modified" } } // normal diff  
  | { error: string };  
  // failure
```

4) Minimal Valid JSON Pair (data + flow)

data.json

```
{
  "nodes": [],
  "runtimes": [
    {
      "EditorID": "MR_1",
      "current_node": { "type": "MR" }
    },
    {
      "EditorID": "OP_1",
      "current_node": { "type": "OP" },
      "tool": { "name": "opensxm_auto" },
      "params": { "path": "Data", "filename": "07_15_25_002.sxm", "pnum": "1" }
    }
  ]
}
```

flow.json

```
{
  "runtime_sequence": ["MR_1", "OP_1"]
}
```

5) Suggested Consistency & Safety Checks

These checks are not enforced in code here but are recommended for robustness:

1. Schema sanity

- Ensure each runtime has `EditorID` (string) and `current_node.type` in {"MR", "OP"}.
- Ensure every `runtime_sequence[i]` exists in `runtimes_by_editor_id`.

2. Status handling

- Decide whether `execute_flow` should `continue` instead of `return` when encountering an already executed runtime.

3. Error channel

- When writing `{"error": ...}` into `runtime_outputs`, consider also setting a separate `runtime_status[runtime_edid] = "failed"` for clearer status reporting via `get_runtime_status()`.

4. Tool meta contract (future)

- Define a stable `meta` structure for tool calls (e.g., `{"script": "...", "params": {...}, "outputs": [...]}`) so `ExecutionManager` and the tool layer remain decoupled.